Intrinsic Value Analyzer

Quantitative Stock Market Asset Analysis Model

Model Card V2.2

2026-05-17

Nilrudra Mukhopadhyay

Document Purpose

The following document serves as a report on the performance of the Intrinsic Value Analyzer model, and as a guide to its structure, intended use, and possible future improvements. Sections of this report include basic model information, training data structure, evaluation process, results, known limitations, and possible improvements.

Model Information

The intrinsic value analyzer is built on the idea of estimating a stock's "real" price based on a formula first developed and introduced by Benjamin Graham. The formula uses two constants, first the P/E ratio of a stock with no growth at 8.5, and the second the average yield of the AAA corporate bond at 4.4. It then expresses the Intrinsic Value as a dollar amount, forecasting the price of the asset after the market corrects itself.

$$V = \frac{EPS \times (8.5 + 2g) \times 4.4}{Y}$$

Figure 1: Benjamin Graham's Intrinsic Value formula

The issue with performing asset valuations with Benjamin Graham's formula alone is that the model breaks down when introduced to non-standard or cyclical businesses, where the cash flow is irregular and the P/E changes with seasonal business downturns. Benjamin Graham's formula inherently requires stocks with continuous stable income and "ideal" circumstances to correctly evaluate their worth. Trained investors with years of practice are able to build intuition which can potentially identify value traps or hidden gems even when the IV says otherwise, but this isn't widely feasible. The best solution is to utilize Machine Learning models capable of computing statistical probabilities within data clusters and building "intuition" mathematically.

In conclusion, to automate the process of discerning if a stock is "overvalued" or "undervalued" from Benjamin Graham's formula a Machine Learning Model can be used. Resulting in a modular software component which can either become part of a larger array of Quantitative models or become the core component of an asset valuation application.

Model Dependencies

To run this packaged model in another script or environment, the following Python ecosystem libraries must be installed as shown below.

- Scikit-learn - specifically for MLPClassifier and StandardScaler
- Joblib - for loading the serialized .joblib model artifact file
- Pandas - when handling data matrices and structural array transformations

Model Inputs

The model expects a 2D array-like structure (a Pandas DataFrame row or a NumPy array) containing exactly three continuous numerical features, all with the float data type as mentioned below.

- pe_ratio - The price-to-earnings ratio of the stock (Accepts $0 values or negative values representing unprofitability)
- iv - The absolute calculated dollar amount of the Benjamin Graham Intrinsic Value, generated by the formula shown in figure 1
- price - The current trading market price of the stock or asset

Crucial Operational Note: when implementing the model any raw inputs must pass through the loaded StandardScaler first via transform() before being handed to the Neural Network.

Model Outputs

Depending on which method you call on the loaded Neural Network model, it produces two separate outputs. The model predict(x_scaled) function outputs a binary integer value.

- 1 represents a stock which the model predicts as “Undervalued” forecasting the price will rise
- 0 represents a stock which the model predicts as “Overvalued” forecasting the price going down

Meanwhile, predict_proba(x_scaled) returns the raw probability distribution for both classes which could be useful for showing the confidence the model has on its predictions.

Model Architecture

The architecture during training is broken into two modular layers, where layer 1 computes the Intrinsic Value as an engineered feature while layer 2 performs value standardization and model training/testing. Below is a breakdown of the layers along with their specific nuances.

1. Layer 1 Feature Engineering: calculates the "Graham” Intrinsic value of the stock, and adds it to the dataframe record alongside the other metrics.
    - Inputs: price, EPS, P/E, expected growth, and AAA corporate bond Yield
    - Outputs: one primary feature the IV expressed as a dollar amount
2. Layer 2 Supervised Learning Model: A Neural Network (Logistic Regressive) trained on Historical data which compares P/E ratios, Graham Intrinsic Values, and the current price against the real outcomes of price movement between the years 2021–2025.
    - Training Label 1 = Winner: the stock grew over the past 4 years
    - Training Label 0 = Loser: the stock fell over the past 4 years
    - Action: the NN learns to "veto" IV results, when the NN sees a historical pattern of "Value Traps" in relation to the P/E ratio and Intrinsic Value.

The model during production consists of a package of artifacts which contain the model, and the model’s StandardScaler. For the most effective forecast, the scaler must be used before feeding the model any new asset metrics, which ensures the model makes predictions on values scaled similarly to its training data.

Training Dataset Sources

Kaggle was the source of all publicly available open datasets used during the development of this model, while the yFinance API (free tier) provided historical and current stock prices, from 2017 to 2021. The yFinance API data was used for calculating compounded annual growth rates and labeling stock growths as “overvalued” or “undervalued”. The kaggle datasets alternatively, were used to extract stock metrics such as names, tickers, prices, P/E, EPS values, and etc. Below are the links to all Kaggle datasets used during the data acquisition process of this project.

US stock index comprehensive data
s&p 500 EPS and PE ratio
Current Corporate AAA bond yield value
s&p 500 growth rate price/returns

Training Data Structure

The data is housed within a local Microsoft SQL Express Database. Interfacing with this database was done through the use of several T-SQL scripts written/executed through SSMS, and python 3 scripts written/executed within a virtual environment using SQLalchemy. Below is the structure SQL used to create the tables for the training data.

```
CREATE TABLE dbo.stocks (
    ticker      VARCHAR(10)   PRIMARY KEY,
    stock_name  VARCHAR(255)  NOT NULL,
    sector      VARCHAR(255)  NOT NULL,
    price       DECIMAL(18,2) NOT NULL,
    earnings    DECIMAL(18,2) NOT NULL,
    pe_ratio    DECIMAL(18,2) NOT NULL,
    market_cap  DECIMAL(20,2) NOT NULL
);

CREATE TABLE dbo.growth_rates (
    ticker          VARCHAR(10),
    start_value     DECIMAL(18, 2) NOT NULL,
    end_value       DECIMAL(18, 2) NOT NULL,
    growth_rate     DECIMAL(18, 2) NOT NULL,
    corp_bond_yield DECIMAL(18, 2) NOT NULL
    CONSTRAINT      fk_stock_growth_rates
        FOREIGN KEY (ticker)
        REFERENCES  dbo.stocks(ticker)
);

CREATE TABLE dbo.historic_valuations (
    ticker          VARCHAR(10),
    valuation       VARCHAR(12),
    CONSTRAINT      fk_historic_valuations
        FOREIGN KEY (ticker)
        REFERENCES  dbo.stocks(ticker)
);
```

Figure 2: Structure T-SQL used to create the tables for model training data

During model training, pulling data into a Jupyter Notebook was done through a combination of the SQL query shown below and a Pandas dataframe.

```
query = """
SELECT
    s.ticker, s.price, s.earnings, s.pe_ratio,
    g.growth_rate, g.corp_bond_yield, h.valuation
FROM dbo.stocks s
JOIN dbo.growth_rates g ON s.ticker = g.ticker
JOIN dbo.historic_valuations h ON s.ticker = h.ticker
WHERE s.price > 0
"""
```

Figure 3: Query string from the model training code

Once the data was pulled into a dataframe, it was organized into the following structure shown below. The real valuations were calculated based on price movement per asset from the years 2021 to 2025, “undervalued” if price went up and “overvalued” if it went down between the date range, assuming the market corrected the prices. Static metrics like EPS, P/E, and etc were taken from the year 2021 to simulate conditions where the model makes predictions in that year. The idea is to have the model think it's predicting during the year 2021 and then move to 2025 to check if it made the predictions correctly. If the stock price moved down from the years 2021 to 2025 but the model during 2021 predicted the asset as being “undervalued” then the model made an incorrect prediction, and vice versa.

ticker	price	earnings	pe_ratio	growth_rate	corp_bond_yield	valuation
string	double	double	double	double	double	integer

Figure 4: Table showing the structure of the dataframe used during model training